

The Problem

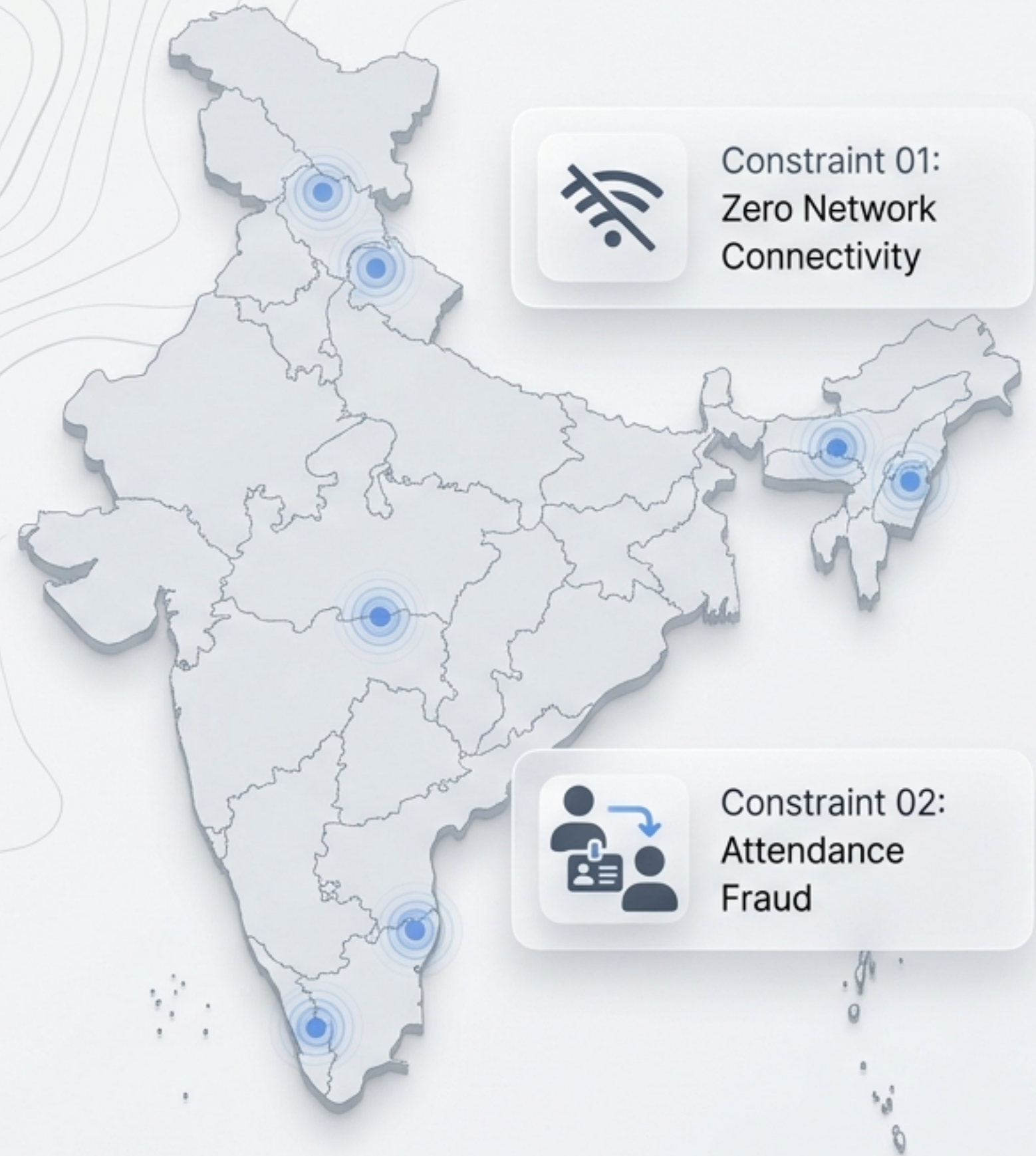
Good morning Judges.

NHAI manages massive highway constructions across India. Your thousands of field engineers work in remote locations—mountains, rural highways, jungles—where there is zero internet connection.

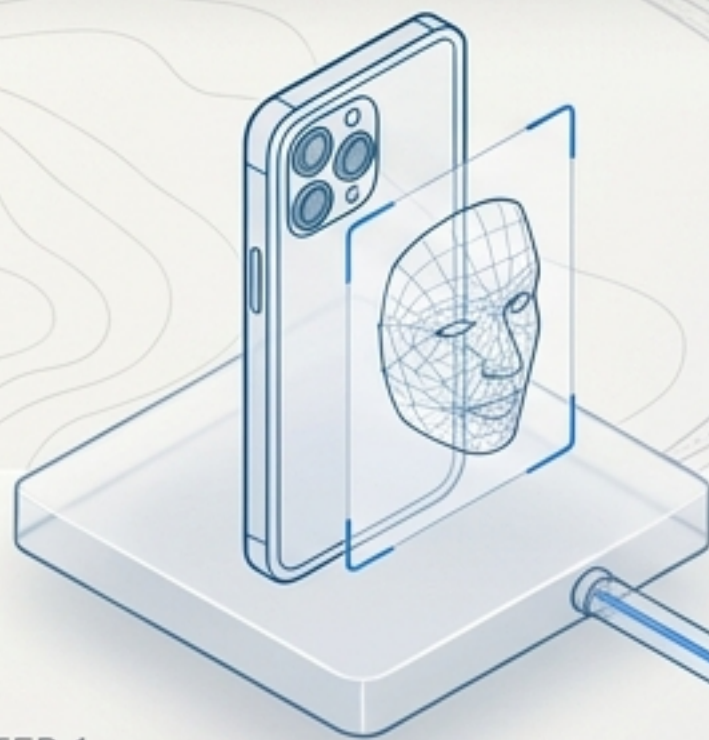
Tracking authentic attendance in these zero-network zones is currently impossible and prone to buddy-punching or attendance fraud.

You need a system that proves exactly who is standing there, proves they are a living human, and does it completely offline.

That is why we built NHAIFaceID.



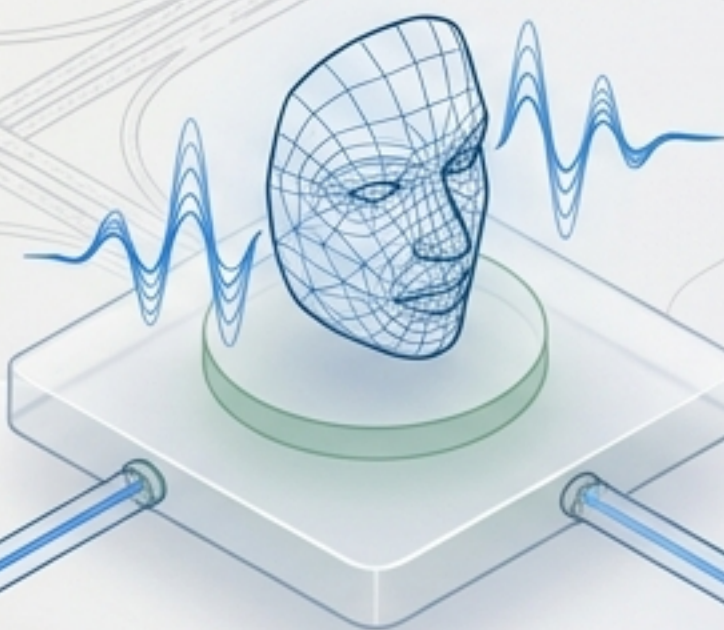
NHAIFaceID is a highly optimized React Native SDK designed to drop perfectly into your existing Datalake 3.0 app. It operates on a robust offline architecture performing three core tasks:



STEP 1

Native camera framing and MediaPipe 3D face mesh detection.

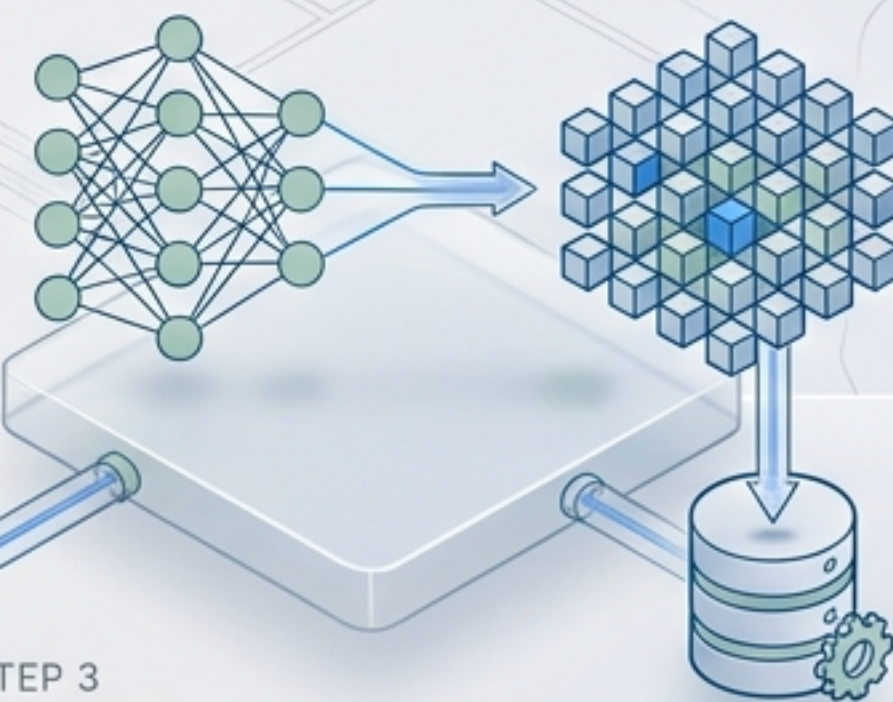
Leverages native platform camera capabilities for precise framing, followed by real-time extraction of a detailed 3D facial landmark mesh using MediaPipe.



STEP 2

A strict Anti-Spoofing Liveness Audit tracking micro-movements to prove the subject is alive.

Implements rigorous real-time analysis of facial dynamics, including blink detection and subtle movements, to prevent spoofing attacks.



STEP 3

Fast execution of a quantized MobileFaceNet AI model to extract a 192-dimensional facial signature and compare it against an offline SQLite database.

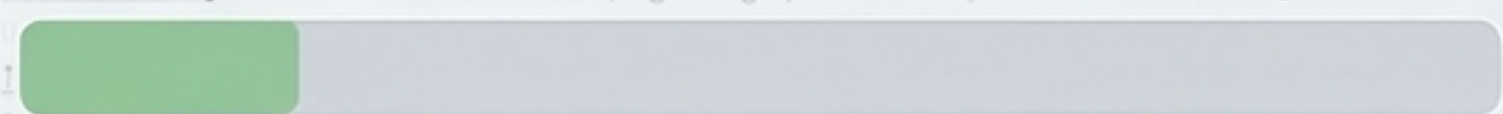
Utilizes an optimized, quantized MobileFaceNet model for efficient local inference, generating a compact 192D signature for rapid comparison with stored identities in an encrypted offline database.

The biggest constraint you gave us was keeping the AI models under 20MB so the Datalake app doesn't become bloated, and executing in under 1 second. By pushing heavy image rotations and bitmap processing down into a custom native Android Kotlin module, and utilizing a highly quantized MobileFaceNet TFLite model, our total bundled AI footprint is under 5 Megabytes. We completely crushed the 20MB limit. Furthermore, our entire offline processing pipeline runs from start to finish in 200 to 400 milliseconds on standard mid-range Android 8+ devices with 3GB of RAM, effortlessly passing your 1-second speed requirement.

The Constraint Dashboard

Title of Hackathon: Develop a highly based secure offline facial recognition, and liveness detection system for remote locations.

AI Footprint Constraint Hackathon Target: 20MB



Actual Achieved: < 5MB

Target Audience: Open to IT Students, A/ML. Enthusiasts, App Developers, etc.

Technical Constraints & Specifications:

1. **Framework Compatibility:** Must be fully compatible with React Native, supporting cross-functional deployment on both Android and iOS.
2. **Model Footprint:** The AI model must be lightweight to avoid bloating the Datalake app package. The target size is ~20MB.
3. **Presentation and Technical Constraints:** The solution should be well presented in a pptx. / .pdf file. No party code is used and source code of the prototype must be shared. No licestass should be required additionally.

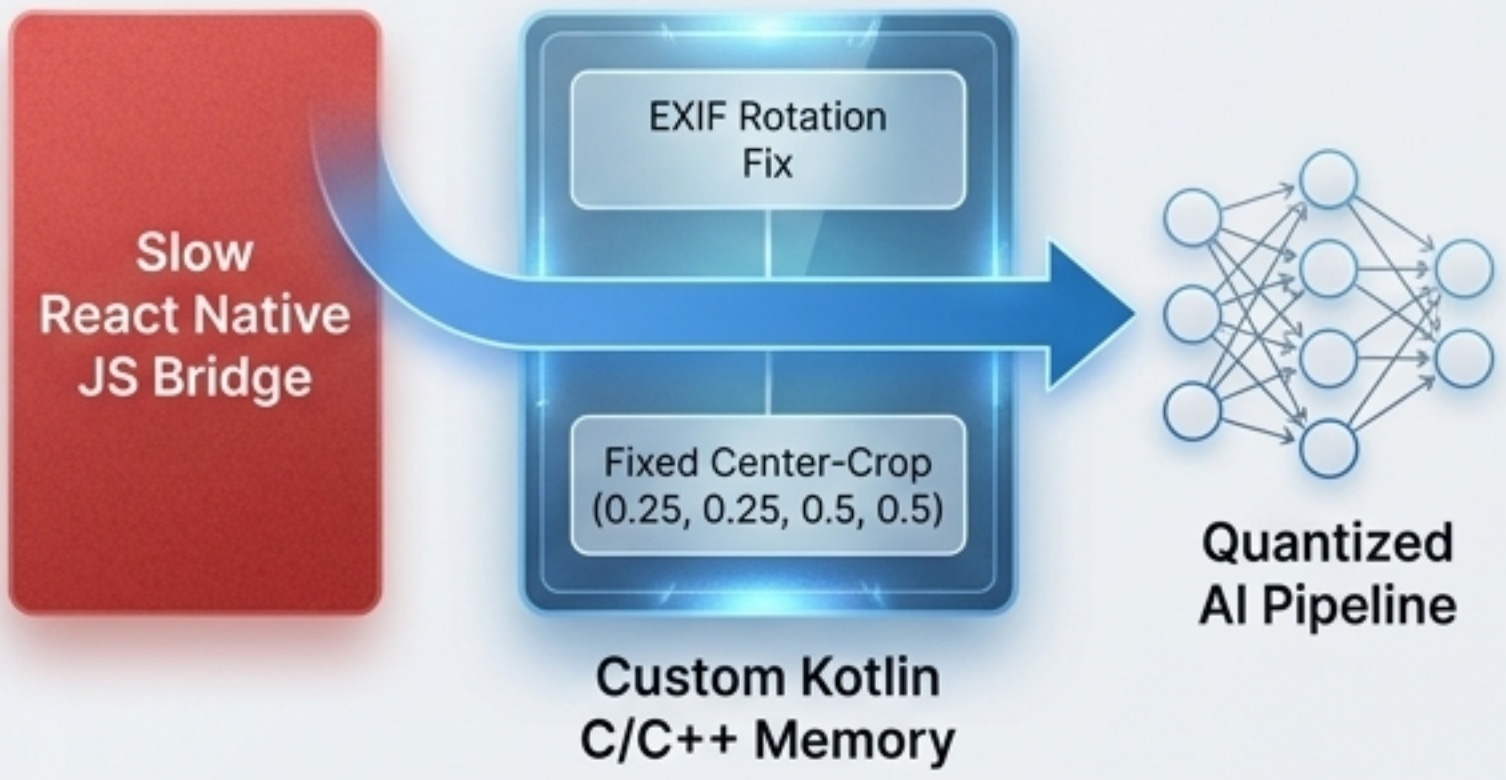
Mandatory Deliverables

1. **Working Prototype with Source Code:** A functional offline prototype (Android + iOS) built in React Native demonstrating the offline capability.



Speed Constraint

The Native Bypass Schematic



Defeating Fraud

How do we stop a worker from just holding up a printed photograph or an iPad video of their friend? Because of our strict offline limits, we engineered a brilliant Micro-Variance Tracking System.

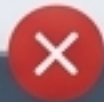
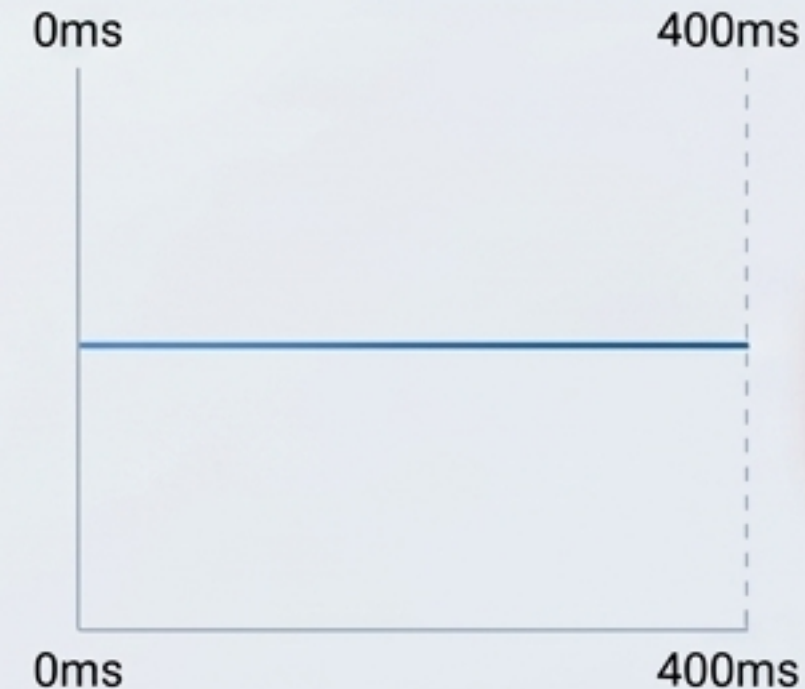
As the worker holds their phone for 400ms, our SDK tracks 468 invisible facial landmarks.

A static photograph or a screen has near-zero variance. If the landmark variance drops below our strict threshold, the system flags it as a Spoof and instantly rejects the scan.

High security using exactly 0MB of extra storage space.



Variance ≥ 0.00012 :
APPROVED



Variance < 0.00012 :
SPOOF REJECTED

Finally, the field engineer **successfully** logs their attendance deep in **a zero-network zone**. They get in their truck and drive back to the city. The absolute second their phone connects to 4G or Wi-Fi, our background network listener wakes up. It silently scoops all the encrypted logs from the local SQLite database, syncs them securely to the **NHAI** AWS servers, and then immediately **purges** the **local data** to comply with storage and privacy limits. Seamless operation with **zero friction**.

SQLite Local Write



Face scanned, 192-D
Cosine Match (Threshold
0.55), encrypted in local DB.

@react-native-community/netinfo



Background listener
actively scanning OS
events for signal.

AWS REST Endpoint



Silent batch transmission
of JSON payloads.

SQL DELETE



Local data is instantly
purged to ensure strict
data retention compliance.

We have packaged this all into a premium, glassmorphism-styled **SDK** requiring just **three** lines of code for the Datalake team to integrate. It is **lightweight**, hyper-fast, highly secure, and built exclusively on open-source technologies. Thank you.



```
import { NHAIFaceSDK } from 'nhai-face-id';  
  
// Drop-in component for Datalake 3.0  
<NHAIFaceSDK onVerify={(data) => syncAttendance(data)} />
```